

CO3-AT1-Assignment

View Only

[← Back](#)

QUESTIONS

Q1

Banking Transaction System –
Exception Handling

10 marks



Q2

Hospital Patient Registration –
User-Defined Exception

10 marks



Q3

E-Commerce Order Processing
– Built-in Exceptions

10 marks



Q4

Online Food Delivery – Multiple
Threads and Execution Order

10 marks



Q5

Manufacturing Plant –
Producer-Consumer Problem

Q5 Manufacturing Plant – Producer-Consumer Problem

10 marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product.

Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:

- Make the producer wait when the buffer is full.
- Make the consumer wait when the buffer is empty.
- Prevent duplicate consumption.
- Ensure that products are consumed only after production.
- Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution

Consumer starts first Consumer waits until a product is available Concurrency / design

10 marks

5/5 answered

Buffer capacity = 1; Products = P1, P2 Producer waits while P1 remains unconsumed Exception / boundary

Your Code

```

1 class Buffer{
2     private String product;
3     private boolean full=false;
4     synchronized void produce(String p) {
5         while(full) {
6             try{
7                 wait();
8             }catch(InterruptedException e) {
9
10            }
11        }
12        product=p;
13        full=true;
14        System.out.println("Produced: "+p);
15        notify();
16    }
17    synchronized void consume() {
18        while(!full) {
19            try{
20                wait();
21            }catch(InterruptedException e) {
22
23            }
24        }
25        System.out.println("Consumed: "+product);
26        full=false;
27        notify();

```

Input

stdin input

Output

Produced: P1
Consumed: P1
Produced: P2
Consumed: P2
Produced: P3
Consumed: P3

```
28     }
29     public static void main(String[] args) {
30         Buffer b=new Buffer();
31         Producer p=new Producer(b);
32         Consumer c=new Consumer(b);
33         c.start();
34         p.start();
35         try{
36             p.join();
37             c.join();
38         }catch(InterruptedException e){
39         }
40     }
41     static class Producer extends Thread{
42         Buffer b;
43         Producer(Buffer b) {
44             this.b=b;
45         }
46         public void run() {
47             b.produce("P1");
48             b.produce("P2");
49             b.produce("P3");
50         }
51     }
52     static class Consumer extends Thread{
53         Buffer b;
54         Consumer(Buffer b) {
55             this.b=b;
56         }
57         public void run() {
58             b.consume();
59             b.consume();
60             b.consume();
```

```
61      }  
62      }  
63 }
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.
